Contents

Future Ideas

Deferred features and improvements that are worth revisiting but not currently prioritized.

Known Critical Bugs

*None currently open. See **Appendix — Resolved & Obsolete** at the end of this doc for fixed entries.*

Journey CTA spotlight — wiring leftovers

The reusable `<Spotlight target={ref} active={...} onDismiss={...} />` component shipped on 2026-04-29 (`landing/src/components/guide/Spotlight.jsx`) and replaces the ad-hoc per-page `xo-spotlight-pulse` toggle. Wired so far:

- **Step 4 (`?action=train-bot`)** — `BotProfilePage` Train button. □
- **Step 5 (`?action=spar`)** — `BotProfilePage` Spar block; `ProfilePage` forwards the action to the bot detail page same way it does for `train-bot`. □

Not yet wired (each one is one `<Spotlight>` render line + a small destination handler):

- **Step 3 (`?action=quick-bot`)** — `QuickBotWizard` "Next" button. The wizard is already a focused modal so the spotlight is lower-value here; skip unless usability tests show the Next button blends in.
- **Step 6 (`?action=cup`)** — Curriculum Cup card. No `?action=cup` handler exists on `ProfilePage` (or anywhere else), and there's no Cup-card destination element to ref. Needs the destination feature first.
- **Step 7 (`?action=cup-result`)** — result row in the tournament list. Same as step 6 — destination doesn't exist yet.

Effort: ~30 minutes per remaining wiring site once the destination handler is in place.

Status snapshot (last reviewed 2026-04-23)

Item	Status
Real-Time Presence / Inactivity Detection	Mostly done (presence store + heartbeat live; away/active refinement open)
Multi-Game Bots (now Phase 3.8)	In-plan (scheduled as Phase 3.8 of the Implementation Plan)
Real-Time Games Against Bots (Pong)	Open (Phase 6)
Persist Game State Through Deploys	Obsolete (replaced by DB-backed `Table` rows in Phase 3.4)
Backend Logs in Admin Log Viewer	Open
Guide Help Subsystem (Chat Interface)	Open
Guide as Navigation (Command Palette)	Open
Configurable Guide	Obsolete (iframe guide retired; premise gone)
Multi-Game Architecture	Largely done as the Game SDK (Phases 1.1–1.4) — remaining games are their own phases
Tier 2/3 instrumentation	Partly done (3 counters live; rest in `doc/Observability_Plan.md`)
Recurring tournaments schema refactor (Phase 3.7a)	Shipped — `TournamentTemplate` + `templateId` FK live; `AdminTemplatesPage` + `AdminTemplateDetailPage` exist (see Appendix)
Tournament admin UX overhaul	Open — operator-facing surfaces around recurring tournaments are clumsy; entry below
`table.released` per-reason soak monitor	Open (post-prod-launch — needs real traffic to be meaningful)

Migration-sensitivity audit (2026-04-23)

Which of the items above actually benefit from shipping *before* prod has real users? Only schema/data-migration costs scale with user volume; UX features cost the same at 0 users or 10,000.

Item	Migration-sensitive?	Verdict
Real-Time Presence	Done ()	—
Multi-Game Bots (Phase 3.8)	No — schema already anticipates it (`BotSkill (botId, gameId)` unique from Phase 1.7)	Do via 3.8 on the normal track
Pong (Phase 6)	No — new subsystems, no data migration	Ship when scheduled
Persist Game State	Obsolete ()	—
Backend Logs → DB	No — just starts writing more rows	Ship anytime
Guide Chat	No — UX feature	Ship anytime
Command Palette	No — UX feature	Ship anytime
Configurable Guide	Obsolete ()	—
Multi-Game Architecture	Done via SDK ()	—
Tier 2/3 instrumentation	No — counters, not schema	Ship anytime
Recurring tournaments refactor	**Yes — template vs occurrence split**	**Phase 3.7a (now)**

Also folded into Phase 3.7a for the same "easier empty than later" reason (not in the original Future_Ideas list):

- Bot `displayName` uniqueness policy
- Public profile URL structure (reserve `/users/:username`)

- OAuth prod redirect URLs at providers
- Seeded built-in bot polish (avatars, bios, ELO ladder)

Multi-Game Bots

What: Allow a single bot to have trained models for multiple games (e.g., XO and chess). Today a bot is effectively XO-only — it holds one model. A multi-game bot would hold one model per supported game and compete on each game's leaderboard independently.

Why deferred: Requires a second game to exist on the platform first. The groundwork is already in place — the credits system is game-agnostic (`appId` field), the `Game` table has an `appId` column, and the Credits Plan explicitly notes "a bot can hold one model per supported game." Bot slot limits already govern agent count, not model count.

What it would take: - Schema: `BotModel` table keyed by `(botId, appId)` to hold per-game weights and ELO separately from the bot's top-level record. - Training UI: game selector when starting a training session in the Gym. - Leaderboard: per-game filtering so a bot's XO ELO and chess ELO are tracked independently. - Community bot matchmaking: players select a game, and only bots with a model for that game appear.

Complexity: Medium-to-large. Straightforward once a second game is added; no point building it for a single game.

Real-Time Games Against Bots (e.g. Pong)

What: Support games with a continuous real-time loop — not turn-based. A classic example is Pong, where the bot controls a paddle and reacts to ball position in real time rather than waiting for a discrete move prompt.

Why deferred: The current architecture is designed around turn-based games (discrete moves, game recorded on completion). Real-time games require a fundamentally different loop: a shared simulation running at a fixed tick rate, input from both sides on every frame, and a bot that acts on continuous state rather than a board snapshot.

What it would take: - **Game loop:** server-authoritative tick loop (e.g. 60Hz) running in the backend, or a client-side loop with the bot running in the browser. Client-side is simpler to start and avoids server compute overhead for solo bot games. - **Bot model:** the AI input is a continuous state vector (ball position, velocity, paddle positions) rather than a discrete board encoding. Suitable algorithms: DQN or Policy Gradient trained on the continuous state space. The existing Gym infrastructure could be extended since DQN already trains on arbitrary state vectors. - **Rendering:** a canvas or WebGL game loop on the frontend replacing the current board grid. - **Recording:** game outcome (win/loss/score) still POSTed to `/games` at completion — credit and ELO hooks unchanged. - **PvP extension:** two human players could also play real-time games against each other via the existing WebSocket infrastructure, with the server relaying inputs rather than authoritative state.

Complexity: Large. The game loop and rendering are new territory. The AI training pipeline is more reusable than it might seem — the Gym's episode-based training maps naturally to a real-time game where each episode is one full match.

Backend Logs in Admin Log Viewer

> **Status update (2026-04-29):** the frontend half landed — `landing/src/lib/frontendLogger.js` batches errors / warnings to `POST /api/v1/logs` and the admin Log Viewer now actually populates with `source: frontend` rows. `setLogUserId` is wired through AppLayout so user context is captured. Backend pino → DB is the remaining piece; the rest of this entry covers what's left.

What: Route backend (pino) logs into the database so the admin Log Viewer shows all four sources — `api`, `realtime`, and `ai` — alongside the frontend rows already flowing in. Currently pino writes to stdout (visible in the Fly.io log stream) but never reaches the `logs` table.

Why deferred: stdout logs are accessible via Fly.io / `docker compose logs` for now. The viewer is already useful for frontend errors. Wiring pino to the DB adds write pressure on every request.

What it would take: - **Pino DB transport:** a custom pino transport (or `pino-transport` wrapper) that batches log entries and inserts them into the `logs` table, respecting the existing `pruneIfNeeded` limit. Use `source: 'api'`, `'realtime'`, or

`'ai'` depending on origin. - **Log level threshold:** only write INFO and above from the backend to avoid flooding the table with debug noise. DEBUG can remain stdout-only. - **Live tail:** backend log entries flow through the existing `appendToStream('admin:logs:entry', ...)` path automatically once they're written via the same POST handler the frontend logger uses (or via a direct stream emit from the transport).

Complexity: Small-to-medium (~half a day). The DB schema, ingestion endpoint, pruning, frontend logger, and live-tail SSE are all in place — the missing piece is just the pino → DB bridge.

Guide Help Subsystem (Chat Interface)

What: Wire up the "Ask Guide anything…" input at the bottom of the Guide panel so users can ask natural-language questions and get contextual answers from an LLM.

Why deferred: The panel footer placeholder exists but is not connected to any backend. Building it well requires deciding on context injection strategy, conversation persistence, and cost controls.

What it would take: - **Backend endpoint:** `POST /api/guide/chat` — accepts `{ message, context }` and streams or returns an LLM response. Context should include the user's journey progress, current page, bot name/config, and recent activity so answers are relevant. - **Conversation state:** local component state (or `guideStore`) to hold message history for the current session. No persistence needed initially. - **UI:** replace the placeholder `div` in `GuidePanel.jsx:153–167` with a real `<textarea>` + send button, and a scrollable message thread above it inside the panel body. - **Rate limiting / cost control:** per-user request throttle on the backend to prevent runaway LLM spend.

Complexity: Medium (~2 days). The panel, orb, and store are all wired up — chat is the only missing piece.

Guide as Navigation System (Command Palette Evolution)

What: Add a ⌘K command palette — a keyboard-invokable search overlay (Spotlight / Linear-style) that lets users jump anywhere in the app by typing rather than clicking through the nav.

Current navigation structure: - **Desktop:** a top header bar with Play, Gym, Puzzles, Rankings as primary links, plus Stats / Profile / About in-line. Admin links appear for admin users. - **Mobile:** a fixed bottom tab bar (Play, Gym, Ranks, Stats, Profile) plus a hamburger menu that expands the full link list including Settings, FAQ, and About. - **Guide button:** a pulsing "Guide" button sits next to the logo in the header and opens the Getting Started modal, whose cards navigate directly to destinations via `target="_top"` links. Users can hide this button in Settings.

The nav works fine but requires knowing where things live. There's no way to reach a page by typing its name, and no single surface that lists every destination at once.

How the palette would work: - Press ⌘K (Ctrl+K on Windows) from anywhere to open a centered overlay with a search input and a list of destinations. - The list pre-populates with the same links in `MENU_LINKS` — Play, Gym, Puzzles, Rankings, Stats, Profile, About, FAQ, Settings — plus admin links when applicable. - Typing filters the list instantly. Enter or clicking an item navigates and closes the palette. Escape closes without navigating. - A small ⌘K hint badge in the header (next to the Guide button) would make it discoverable.

Relationship to the guide: The guide is visual and onboarding-oriented — it shows the journey from new user to competitor. The palette is speed-oriented for returning users who already know what they want. They serve different moments and can coexist.

Why deferred: The existing nav covers current usage. The palette pays off most when users are frequent enough to remember keyboard shortcuts.

Complexity: Medium (~2 days). Purely frontend — a new React component with a `keydown` listener at the app root, no backend changes needed.

Tier 2/3 transport — instrumentation (partly done)

The first three items (SSE client count, presence-store size, XREAD-loop heartbeat) are wired in `resourceCounters.js`. The remaining items below are tracked more fully in **`doc/Observability_Plan.md`** (SSE broker peak/age, Redis stream XLEN + consumer lag, Web Push delivery metrics). Treat this entry as the short form; work from the Observability Plan when picking up an observability sprint.

Open nice-to-haves — wire them in once real traffic lands or when push starts behaving oddly:

- **Push subscriptions count** — snapshot `db.pushSubscription.count()` to see how many device endpoints we're pushing to. Also useful for sizing UI in the admin health dashboard.
- **Push send metrics** — expose counters from `pushService`: `pushSent`, `pushFailed` (transient, non-404/410), `pushPurged` (dead endpoints). Surfaces success-rate and catches a VAPID misconfiguration quickly.
- **Redis Stream length** — `XLEN events:tier2:stream`. Bounded by MAXLEN=5000 so it'll always cap there, but tracking the value confirms trimming is working and gives a rough "events per minute" signal when cross-referenced with snapshot timestamps.

Low priority, mentioned for completeness:

- **`/api/v1/presence/heartbeat` QPS** — normal load is `(online users) / 15s`. A sudden 10× spike indicates a client-side retry-loop bug.
- **Dispatch → push fan-out counter** — how often `notificationBus.dispatch` actually lands a push vs skips because SSE was online. Useful for tuning which event types should have `push: true` in the REGISTRY.

Effort: each item is ~5–10 LOC in `resourceCounters.js` plus a small `export function` in the owning module (`pushService.js`, `tournament/src/lib/redis.js` for XLEN, etc.).

Tournament admin UX overhaul

The schema refactor (Phase 3.7a — `TournamentTemplate` + `templateId`) shipped: see the Appendix entry. What did **not** ship is the operator-facing UX that takes advantage of the cleaner model. Today the create/edit/manage flow for recurring tournaments is clumsy in nine concrete ways, all of which an admin runs into within their first few minutes of use.

Symptoms of the clumsiness (observed on prod 2026-05-09):

1. **One form does both jobs.** `landing/src/components/tournament/TournamentForm.jsx` (431 lines) creates a one-off tournament *and* a recurring template by toggling `isRecurring`. Recurrence config (interval, end date, pause, opt-out) is buried in a sub-section near the bottom of a single-column form.
2. **No schedule preview.** After creating a weekly template, nothing shows "next runs: Wed Apr 22, Wed Apr 29, Wed May 6 …". Admins can't sanity-check the schedule without waiting for the sweep to spawn the next occurrence.
3. **Looking up a recurring series is broken.** `RecurringRegistrations` (`AdminTournamentsPage.jsx:664`) requires the admin to **paste a Template ID** into a text input. There's no clickthrough from the tournaments list to its registrations.
4. **Pause / Stop / Cancel-this-occurrence semantics aren't surfaced.** `recurrencePaused` is a checkbox inside the edit modal. There's no row-level "Pause series" / "Stop series" action; admins conflate cancelling one occurrence with stopping the whole thing.
5. **No edit-template-vs-edit-occurrence branching in the UI.** The schema separates them, but the form doesn't visually distinguish "you are editing the template (affects all future runs)" from "you are editing this occurrence."
6. **Seed bots and standing human registrations are managed in a separate panel.** Most "I want a weekly cup with these 4 bots" workflows take 3 surfaces — create form, seed-bots tab, registrations panel — when they should be one.
7. **No clone affordance.** Common operator action ("make another one like this on a different day") requires retyping every field.
8. **Date input inconsistency.** `startTime` uses the custom `DateTimePicker` (Safari-safe); `recurrenceEndDate` is a bare `<input type="date">` with the just-shipped "no end date" toggle as a workaround. The fix is to drop bare native date inputs entirely and use the same picker.
9. **No timezone affordance.** Datetimes are entered in browser-local time with no display of the resolved UTC or "tournament starts 14:00 UTC for users in TZ X."

Plus a structural near-miss: `isTest` and `isRecurring` are 150 lines apart in the form — creating a test recurring series for QA is undocumented.

Proposed scope (do as a single sprint):

- **(a)** Recurring-series list view at `/admin/templates` with status pills (active / paused / stopped), next-run timestamp, subscriber count, seed-bot count. Click row → template detail page.
- **(b)** Schedule preview on the template detail page — render the next 5 occurrences (computed from `recurrenceInterval` + `recurrenceEndDate` + `recurrencePaused`) so admins can verify the schedule visually.
- **(c)** Edit-template vs edit-occurrence branching. The current `TournamentModal` should detect `templateId` and route to a "Template editor" mode with explicit "this affects all future runs" messaging; otherwise the existing single-occurrence edit.
- **(d)** Row-level actions on the templates list: **Pause series**, **Resume series**, **Stop series**, **Clone**. Mirror these on the template detail page.
- **(e)** Embedded standing-registration management — a panel inside the template detail page (not a separate lookup). Add/remove humans, view missed-counts, opt-out users.
- **(f)** Embedded seed-bot management — same idea: a panel on the template detail page, not a separate tab.
- **(g)** Clone-from action on every tournament row in `AdminTournamentsPage` — pre-fills `TournamentForm` with the source's config, clears name + dates.
- **(h)** Replace every bare `<input type="date">` with the existing `DateTimePicker`. Add a small TZ display next to start times (e.g. "14:00 ET / 18:00 UTC").
- **(i)** Reorganise `TournamentForm` into three top-level groups: **Basics** (name, game, mode, format), **Schedule** (start, registration, recurrence), **Advanced** (seed bots, pace, opt-out, isTest). Recurrence becomes a first-class section, not a buried sub-section.

Effort: Medium-large (~3–5 days). Most of it is React surfaces; the backend is fine. Suggest tackling in the order above (a→i) so each piece is independently shippable.

Planned foundation sprint (~1.5–2 days): `(h) → (i) → (c)` — date picker consistency, then `TournamentForm` reorg into Basics / Schedule / Advanced, then the template-vs-occurrence edit-mode branching. Items (a) and (f) are already shipped (the template pages + `SeedBotsPanel` exist); (b)(d)(e)(g) become parallel leaves once (c) lands.

Why now (vs deferred again): the schema refactor shipped without the matching operator UX, so admins have been running on the worse-of-both-worlds — new schema complexity, old single-form ergonomics. The Cora-3 / Cora-1 admin flows from 2026-05-09 surfaced this concretely.

`table.released` per-reason soak monitor — □ OPEN (defer until prod has traffic)

Background: Chunk 3 of the table-fixes sweep added a per-reason `table.released` counter to `/api/v1/admin/health/tables` (reasons: `disconnect`, `leave`, `game-end`, `gc-stale`, `gc-idle`, `admin`, `guest-cleanup`, plus `OTHER` catch-all). The shape of the per-reason histogram is the V1-acceptance success metric for "where do tables actually die" — does the disconnect bucket dominate (Safari hang regression) vs. the game-end bucket (healthy completion), is the `OTHER` bucket nonzero (typo'd reason at a call site), is `gc-idle` climbing (idle abandonment runaway), etc.

Why deferred: On staging the only traffic is manual QA — the per-reason distribution reflects the tester's clicks, not real user behaviour. Running a soak there would just measure the test, not the system. The metric's value scales with traffic.

What to do post-prod:

- Schedule a periodic poll of `/api/v1/admin/health/tables` (e.g. hourly via the same scheduler used for tournament sweeps, or a cron-driven Slack/Linear post). Diff against the previous reading and post the per-reason deltas.
- Alert thresholds (rough first cuts; tune from data):
  - `OTHER > 0` for any window → call-site typo, page on-call.
  - `disconnect / game-end > 0.5` over a 1-h window → Safari/network regression suspected; page on-call.
  - `gc-idle` rising > 5/hour while active sessions are non-zero → idle threshold misconfigured.
- Cross-reference with `tableCreateErrors.P2002` (should be ~0 post-chunk-1) and `gc.secondsSinceLastSuccess` (should be < 600s).

Effort: ~2 hours. Reuse the existing scheduler + a small `lib/healthDiff.js` to compute deltas. Pairs naturally with the rest of the Tier 2/3 instrumentation work (see entry above).

Redis-backed `sseSessions` registry — □ OPEN (defer until non-Fly hosting is on the table)

Background: SSE sessions are stored in a per-process `Map` in `backend/src/realtime/sseSessions.js`. With more than one backend machine, follow-up POSTs round-robin via the LB and ~50% land on the machine that doesn't have the session, returning 409 `SSE_SESSION_EXPIRED`. Surfaced on prod 2026-05-04 when prod scaled past 1 backend machine — staging was unaffected because it runs a single machine.

What we shipped instead (2026-05-04): `backend/src/realtime/flyReplay.js` — session ids are minted as `<FLY_MACHINE_ID>.<nanoid>` and `requireSseSession` emits a `Fly-Replay: instance=<owner>` header when a POST lands on a non-owning machine. Fly's edge proxy transparently replays the request on the right machine. Off-Fly (local dev / tests), `FLY_MACHINE_ID` is unset and the path is dormant — sessions are bare nanoids and behavior matches the original sync API.

Why Fly-Replay was the right call now: the actual `res` writable for an open SSE connection lives in one machine's process memory and cannot be migrated. Cross-machine *event delivery* already works today via the redis-streams broker (each machine subscribes and pushes to its own connected clients). The only thing that needs cross-machine coordination is the *session-liveness lookup* — and Fly-Replay routes that lookup back to the connection-owning machine in ~10ms with zero state migration. A Redis-backed registry would solve the same lookup with ~500 LOC of changes (async API ripples through 12 test files + 24 callsites). Until we have a concrete reason to leave Fly, the smaller fix is correct.

When to do the Redis migration:

Trigger on any of:

1. **Non-Fly hosting decision** — moving any backend instance to a non-Fly target (AWS/GCP/Cloudflare/self-hosted K8s) where `Fly-Replay` doesn't exist. Prerequisite for the move, not an after-the-fact fix.
2. **Multi-cloud / multi-provider deployment** — running backend simultaneously on Fly + another platform for redundancy or geo. Fly-Replay only routes within Fly, so cross-provider sessions need a portable lookup layer.
3. **Fly deprecates or rate-limits Fly-Replay** — vendor risk; if the header behavior changes, we need an exit.
4. **Replay-tax becomes a measurable problem** — if backend p95 baselines show the ~10-20ms replay penalty is dominating a hot endpoint and we want to eliminate it, redis lookup on every machine removes the round-trip. Unlikely to matter at our scale, but worth re-checking annually.

What to build (when triggered):

- Move `_sessions` Map to a Redis hash `sse:session:<id>` with 60s TTL, refreshed every `touch()`.
- Move `_byUser` Map to a Redis set `sse:byuser:<userId>`, expired with the parent.
- Keep `_pendingDispose` timers and `_onDispose` callbacks in-memory on the originating machine (they fire off the connection-close event, which always happens locally).
- Make `get`, `forUser`, `joinTable`, `leaveTable`, `touch`, `tablesFor`, `pongRoomsFor` async.
- Add a Lua script (or pipelined commands) for the read-modify-write of `joinedTables` to avoid races between concurrent POSTs on different machines.
- Tear down `flyReplay.js` and revert `events.js` / `realtime.js` edits — Fly-Replay becomes dead code at that point.

Effort estimate: 2-3 days for the rewrite + test updates, plus 1 day of staging soak before promote. Pair it with the move to whichever new hosting target triggers it — the work is mostly the same.

Doc cross-refs: `backend/src/realtime/flyReplay.js` (current implementation), `doc/Realtime_Channels.md` (channel namespace + POST routes affected).

Appendix — Resolved & Obsolete

Entries that were once "future ideas" or open bugs but have since been fixed, superseded, or rendered obsolete. Kept for archaeology — they often explain *why* the current architecture looks the way it does. Newest at top.

□ Bot Challenge & Discovery — shipped 2026-05-10

Problem: "play another player's bot" was the platform's stated key differentiator but had no UX path. `POST /rt/tables { kind: 'hvb', botUserId }` and `GET /api/v1/bots` already supported it server-side; the gap was discovery + a one-click action surface.

What shipped (Phases A → C):

- **Primitives** (Phase A): `useBots()` SWR hook over the public bot list, `<BotCard>`, `<ChallengeButton>` (default + icon variants, guest-friendly, posts `/rt/tables` and navigates to `/play?join=<slug>`), `<BotFilterBar>` (search, ELO range, owner toggle, game select, "show all bots" toggle).
- **Surfaces** (Phase B): new `/bots` directory (scrollable `ListTable` of Bot/Owner/ELO/Games/Play); "Challenge" sections on `BotProfilePage` and the `/rankings` bot rows; "Challenge any bot" link on Home below "Watch another match"; About moved out of primary nav into the footer, replaced by Bots.
- **Pickers** (Phase C): tabbed bot picker in the Create Table modal (My / Community / All); Quick Match button on the directory header — calls new `GET /api/v1/bots/quick-match` (guest-OK, ELO-window candidate picker that excludes own bots).
- **Skill-gating:** backend `listBots()` now returns `playableGameIds` per bot, sourced strictly from `BotSkill` rows (legacy `botModelType='minimax'` bots without a skill row were false-positives — they reject at table-create with `NO_SKILL`). The directory filters to playable bots by default; a "Show all bots" toggle reveals the rest with a greyed-out Challenge button and a "No skill for XO" hint.
- **Context-aware navigation:** every linker to `/bots/:id` (directory rows, Rankings, ProfilePage's "My bots") passes `state.from`; `BotProfilePage`'s back link prefers state with a `/bots` fallback. `ChallengeButton`/`QuickMatchButton` thread the same state into `/play`; `PlayPage.leaveHref` honors it so "Leave Table" returns the user to the directory they came from instead of a hardcoded `/tables`.
- **Guest path:** every challenge surface works for unauthenticated visitors; the existing post-game signup CTA fires after the first finished game, not as a gate.
- **Tests:** unit coverage on each primitive + page + the quick-match route; new `e2e/tests/bot-challenge-flow.spec.js` covers the guest path through directory, profile, Quick Match, and Rankings.

Original plan doc: `Bot_Challenge_Plan.md` (deleted on completion — this entry is the digest).

□ Bot best-of-N series cap mis-formula — fixed 2026-05-09

Symptom: every prod tournament with bot-vs-bot matches came in 10–35% over its theoretical bracket-game ceiling. Cora 3 (6-team SINGLE_ELIM bestOf3, all bots) played 20 games against an expected 15 — three of five played matches recorded `p1Wins=0, p2Wins=0` despite COMPLETED status, and showed exactly 5 games each (all draws + deterministic-tiebreaker resolution).

Root cause: `backend/src/realtime/botGameRunner.js:230` had `const hardCap = Math.max(bestOfN * 2 - 1, 1)`. The `2N - 1` formula is correct for the *other* bestOf convention (where N is the wins required and `2N - 1` is the max games — e.g. "best of 3 wins" = max 5 games). But this codebase uses `bestOfN` to mean **max games** (`winsNeeded = ⌈N/2⌉` a few lines above), so the cap should just be N. For bestOf3 the cap was 5 games instead of 3. Two minimax bots playing optimally always draw → series ran the full 5-draw stretch every time before the deterministic tiebreaker resolved it.

Fix: extracted `seriesGameCap(bestOfN) = max(bestOfN, 1)` and used it. Existing tests only covered `bestOfN: 1`, so the broken path was never exercised. Added 4 new unit-test assertions covering bestOf1/3/5 + nullish-fallback. Bot-vs-bot tournaments now run noticeably faster and tournament game counts match the bracket math used by `expectedGameCount` and the runaway-loop guard.

□ Pending PVP match registry per-process race — fixed 2026-05-09

Symptom: in a MIXED tournament with two human players paired in a round-1 HvH match, the second human's "Play Match" click could return "Table closed due to inactivity" — surfacing as `useGameSDK setAbandoned({reason:'stale'})` from a 404 on `POST /api/v1/rt/tournaments/matches/:id/table`. Single-machine staging never reproduced; prod intermittently did.

Root cause: `_pendingPvpMatches` in `backend/src/lib/tournamentBridge.js` was a per-process in-memory `Map`, populated from a Redis **pub/sub** subscription (not a stream — pub/sub does not replay missed messages). On multi-machine prod, three failure modes diverged the maps:

1. **Autoscale-up** — a new backend machine spawned after `tournament:match:ready` was published never received the event; its map stayed empty.
2. **Rolling deploy** — a redeployed machine lost its in-memory map; existing tournaments lost their entries on the restarted node.
3. **Transient Redis disconnect** — any subscriber gap during the publish missed the event.

`/rt/*` POSTs are routed via Fly-Replay to the SSE-owning machine. If user-1 and user-2's SSE sessions landed on different machines, and user-2's machine didn't have the entry, the claim returned `NOT_FOUND` → 404.

Fix: replaced the in-memory `Map` with a Redis hash `tournament:pending:<matchId>` (TTL via `EXPIRE`, no JS-side prune needed). Falls back to in-memory when `REDIS_URL` is unset (tests / local-only). All four operations became async; six caller sites await them. Added 9 unit tests covering get/set/delete/count round-trips, slug-only updates, null-clear-on-disconnect, missing-key no-op, and bestOfN type coercion.

Verified on 2-machine staging: pre-fix the same shape caught 2/10 failures during a new-machine warmup window. Post-fix: 25/25 across 25 runs on a fresh 2-machine deploy. The race window structurally closes — there is no per-process state to diverge.

□ Recurring tournaments — template/occurrence schema refactor — shipped (Phase 3.7a)

Original problem: a recurring tournament was modelled as a single `Tournament` row with `isRecurring: true` that *also ran as the first occurrence*. The template was both a configuration row *and* a historical record of the first run, and "what recurring series exist?" couldn't be answered without filtering. Admins also conflated "cancel this occurrence" with "stop the whole series."

What shipped (Phase 3.7a):

- New `TournamentTemplate` table holds the recurrence config (interval, end date, paused, seed bots, human subscriptions). It never itself runs.
- Every `Tournament` row is now a pure occurrence with `templateId?: string`. The schema removed `isRecurring` / `recurrenceInterval` / `recurrencePaused` from `Tournament`.
- New `TournamentTemplateSeedBot` and `TournamentRecurringRegistration` tables key on `templateId`, not the first occurrence's tournament id.
- Admin surfaces shipped: `landing/src/pages/admin/AdminTemplatesPage.jsx` + `AdminTemplateDetailPage.jsx`. The recurring scheduler in the tournament service was rewritten to advance from the template, not the first row.

What this entry left behind for the operator UX: see the open "Tournament admin UX overhaul" entry above. The schema landed cleanly; the operator-facing surfaces around it (schedule preview, embedded seed/registration mgmt, edit-template-vs-occurrence branching, clone, TZ display, …) did not. That work is tracked separately.

□ Tournament bot matches stuck IN_PROGRESS — fixed 2026-05-05

Original symptom: every Curriculum Cup on staging stuck `IN_PROGRESS` with `updatedAt === createdAt` and `p1Wins=p2Wins=0`. Bot games actually completed in-memory but the bracket never advanced; journey step 7 never fired; 4 cup-soak e2e suites timed out at 240s.

Root cause: `TOURNAMENT_SERVICE_URL` secret on staging was set to `http://xo-tournament-staging.flycast:3001`. `.flycast` is Fly's anycast private DNS, but only resolves when the target app has a private IPv6 IP allocated (`fly ips allocate-v6 --private`). Our deploy doesn't allocate one, so DNS lookups returned `ENOTFOUND`, every cup match completion POST silently failed (errors logged at `warn` then swallowed), and the bracket-advance pipeline never ran. Prod was using `https://xo-tournament-prod.fly.dev` (public hop) so it worked but at extra latency.

Fix: flip both backend services to `http://xo-<env>-tournament.internal:3001` (Fly 6PN private DNS — resolves without any IP allocation, same private network as `.flycast` would have provided). Verified: a clean cup against staging now completes in ~1 minute and step 7 credits. Runbook updated (`Prod_Bringup_Runbook.md` §xo-backend-prod) so this doesn't recur.

□ Bot model half-converted state after train-guided — fixed 2026-05-05

Symptom: after `POST /api/v1/bots/:id/train-guided/finalize`, the bot ended up with `botModelType: 'minimax'` (unchanged) but `botModelId: <BotSkill UUID>` — half-way between Quick Bot and trained ML bot. Caused journey step-4 e2e tests (`guide-onboarding`, `guide-ui-states`, `journey-train-modal`) to fail asserting `botModelType === 'qlearning'`.

Root cause: `mlService.js#repointBotPrimarySkill` (and the duplicate in `skillService.js`) updated `User.botModelId` when training completed but didn't touch `botModelType`. The finalize handler at `bots.js:469` then saw `bot.botModelId === skillId` (already aligned) and skipped the qlearning flip. Verified on staging — DB rows for recent train-guided bots showed exactly this state.

Fix: both `repointBotPrimarySkill` functions now read `algorithm` from the BotSkill row and write `botModelType` derived

from it (lower-snake-no-underscore: `Q_LEARNING → qlearning`, `MONTE_CARLO → montecarlo`, etc.). Backwards-compatible — no botModelType write when algorithm is null. 7 new unit tests across mlService + skillService.

✅ E2E journey suite cross-origin auth — fixed 2026-05-05

Original symptom: `e2e/tests/guide-onboarding.spec.js` (and 8 sibling specs) failed immediately on staging/prod with `No token in response — user may not be signed in on this context` from `helpers.js:111`. Test signed up on `xo-landing-*.fly.dev` (cookie on landing origin); `fetchAuthToken` then hit `${BACKEND_URL}/api/token` on `xo-backend-*.fly.dev` — different subdomain → no cookie → 401.

Fix: route every test API call through the landing host so the existing `landing/server.js` proxy (`pathFilter: ['/api', '/socket.io']`) carries the cookie + forwards to backend. Replaced `BACKEND_URL` with `LANDING_URL` across all 9 affected specs (88 references). Backend CORS already allows the landing origin via `FRONTEND_URL`; no backend changes needed. The working pattern was already proven in `smoke.journey.spec.js`.

Files patched: `journey-cup` · `guide-hook` · `guide-onboarding` · `guide-ui-states` · `guide-curriculum` · `journey-spar` · `journey-train-modal` · `journey-spotlight` · `guide-phase0`.

✅ Downstream journey-suite issues (post-cross-origin) — fixed 2026-05-05

After the cross-origin auth fix unblocked the suite, several smaller test/staging issues surfaced and were resolved in the same sprint:

- **`guide-hook` step 1 prefs 4xx** — `GET /api/v1/guide/preferences` 404'd because the BetterAuth → application User row hadn't been mirrored yet. Fix: explicit `POST /users/sync` after `fetchAuthToken`, matching the existing pattern in `guide-onboarding`/`journey-cup`.
- **`guide-hook` step 2 demo-watch credit timing** — same race; same `/users/sync` fix.
- **`guide-hook` private-table assertion** — test asserted demo `Table` rows must NOT appear in default `GET /api/v1/tables` for the creator, but backend's documented behavior is "authed users see their own private tables in default" (see `tables.js:276`). Replaced with the right privacy check: an anonymous viewer must not see the demo.
- **`guide-onboarding` step 4 model-type** — covered above (the `repointBotPrimarySkill` fix).
- **Stale slug regex** — `guide-hook.spec.js:80` and `guide-curriculum.spec.js:113` asserted `/^mt-/` but backend uses `nanoid(8)` (no prefix). Updated to `/^[A-Za-z0-9_-]{8}$/`.
- **Username collisions** — hardcoded display names (`Hook Demo`, `Phase Zero`, `Curr Spar`, etc.) collided on the lower-snake-cased username unique constraint after the first staging run. Added `uniqueName(label)` helpers that suffix a random tag.

After these fixes plus the cross-origin and `repointBotPrimarySkill` work, the full journey suite (25 tests across 9 specs) runs green end-to-end on staging.

✅ Real-Time Presence / Inactivity Detection — mostly done

Status: The core heartbeat-based presence was built during the Phase E tier-2 comms work. `backend/src/lib/presenceStore.js` tracks `onlineAt` with TTL; `landing/src/lib/useHeartbeat.js` is the client hook; `GET /api/v1/presence/online` + the `presence:changed` SSE channel expose the state.

What's still open (small refinement, not blocking): the hide/show behaviour uses default visibility — when a tab backgrounds, the heartbeat stops and the server evicts the user after the TTL. There's no explicit `user:away` transition and no distinction between "tab hidden" (may come back in seconds) and "tab closed" (likely gone for the day). Good enough for the admin "online" indicator in its current form.

✅ Multi-Game Architecture — largely done (as the Game SDK)

Status: The Game Adapter pattern proposed here was implemented as the `GameSDK` contract in Platform Phases 1.1–1.4. `packages/sdk` defines `GameMeta`, `GameSDK`, and `botInterface`; `@callidity/game-xo` is the reference implementation; `PlatformShell` loads any `GameMeta`-conforming package via `React.lazy`. `roomManager` was retired in Phase 3.4 — `Table` rows + `previewState` + SDK adapters replaced it.

What's left (tracked in `doc/Platform_Implementation_Plan.md`): - **Phase 4** — Connect4 (2p sit-down, validates the abstraction with a second game) - **Phase 5** — Poker (adds hidden-info via `getPlayerState`, variable player counts) - **Phase 6** —

Pong (adds `tableArchetype: 'head-to-head'` + real-time loop)

The original analysis below remains a useful reference for the per-game rendering strategy (React vs Framer Motion vs Phaser), but the high-level adapter/registry work is done. Leaving below:

What: Evolve the platform to support additional game types — Connect 4, Checkers, card games, and real-time games like Pong — without rewriting the infrastructure that already works.

What's already generic: The room lifecycle (create/join/disconnect/reconnect/close), socket event envelope (`game:start`, `game:moved`), ELO system, game recording, mountain name rooms, spectator system, and credits (`appId` already on the schema) are all game-agnostic today. They need no changes to support new games.

What's XO-hardcoded today: `board: Array(9).fill(null)`, `makeMove({ cellIndex })`, `getWinner`/`WIN_LINES`/`isBoardFull` called directly inside `roomManager` and `botGameRunner`, `playerMarks: X|O`, and `winLine`. `GameBoard.jsx` (~600 lines) and `gameStore.js` are XO-specific. The AI registry calls `aiImpl.move(board, difficulty, currentTurn)` — an XO-shaped interface.

The core abstraction: a Game Adapter

Each game type registers an adapter that owns its rules. The room manager and bot runner stop knowing anything about game logic and delegate to it:

```
{
  appId: 'connect4',
  initialState()                        // returns a fresh game state
  applyMove(state, move)                // returns { state, terminal, winner }
  validateMove(state, move, playerMark)  // boolean
  serializeForClient(state)             // what gets emitted over the socket
}
```

`roomManager.makeMove()` currently calls `getWinner(room.board)` directly. With adapters it becomes `adapter.applyMove(room.gameSta` `move)`. The room carries a `gameType` field and the manager looks up the adapter from a registry — the same pattern as the existing AI registry. On the frontend, `GameBoard.jsx` splits into a generic `GameContainer` (socket connection, room management, scores, forfeit, spectator mode) and a game-specific renderer (`XOBoard`, `Connect4Board`, etc.) that receives standardized state and emits standardized moves.

The four game types and what each requires:

- **Connect 4** — most similar to XO. Different board shape (7×6), gravity mechanic (move is a column index, not a cell index), 4-in-a-row win detection. Fits the adapter interface cleanly. Good first target for proving the abstraction.
- **Checkers** — still turn-based and discrete, but move validation is complex (forced captures, multi-jump chains, kinging). The adapter interface handles it — `validateMove` and `applyMove` just do more work. Socket model is unchanged because state is fully visible to both players.
- **Card games** — turn-based discrete moves, but with hidden information (hand cards). The current socket broadcast model breaks here: the server can't emit full state to the room because each player should only see their own hand. The adapter interface needs a `serializeForPlayer(state, playerMark)` method, and the socket layer must emit per-player views (`io.to(socketId).emit()`) instead of broadcasting to the room. This is the meaningful socket architecture change for card games.
- **Pong / real-time** — fundamentally different. No turns, no discrete moves. Needs a `RealtimeGameRunner` with a server-authoritative tick loop (or client-side simulation with the server recording the final result). The existing `BotGameRunner._runGameLoop` async loop is the conceptual ancestor but would need to run at ~60Hz and push continuous state. See also: *Real-Time Games Against Bots* entry above.

Rendering strategy and Phaser:

No single renderer fits all game types:

Game	Renderer
XO	React (existing)
Connect 4	React + CSS transitions

Game	Renderer
Checkers	React + CSS transitions
Card games	React + Framer Motion
Pong / real-time	Phaser (lazy-loaded)

Phaser is a complete 2D game framework (WebGL/canvas, physics engine, 60Hz game loop, sprite management, input handling). It operates outside React's DOM model — you mount it imperatively in a `useEffect` and tear it down on cleanup. It's the right tool for real-time physics games (Pong) where it saves significant manual work on collision, velocity, and game loop management. It's overkill for turn-based games.

PixiJS is a lighter alternative (~400KB vs Phaser's ~1MB+): WebGL rendering without the physics engine. Better fit if a game needs smooth sprite rendering but not physics — certain card game animations, animated boards.

Critical: Phaser (or PixiJS) must be a **lazy-loaded, per-game dependency** — not a platform-wide import. A player loading Connect 4 should never download Phaser. The game adapter architecture supports this naturally: each game's renderer is its own bundle chunk, loaded only when that game is selected.

Recommended evolution path:

1. **Phase 1 — Extract and prove the adapter (Connect 4):** Move XO logic out of `roomManager` and `botGameRunner` into `XOGameAdapter`. Create the `gameAdapters` registry. Add `Connect4GameAdapter` and `Connect4Board.jsx`. Split `GameBoard.jsx` into `GameContainer` + `XOBoard`. This validates the abstraction without breaking anything.
2. **Phase 2 — Checkers:** Adapter interface unchanged; more complex `validateMove`/`applyMove`. No socket changes.
3. **Phase 3 — Card games:** Add `serializeForPlayer` to the adapter interface. Add per-player socket emission to the socket layer.
4. **Phase 4 — Real-time (Pong):** Separate architecture path. `RealtimeGameRunner`, canvas renderer via Phaser, tick loop. Plan independently once at least one more turn-based game exists.

Complexity: Phase 1 is medium (~3–4 days — the adapter extraction plus a working Connect 4). Each subsequent phase builds on it. The real-time phase is large and largely independent.

□ Persist Game State Through Deploys (Redis-backed Rooms) — obsolete

Status: Superseded by Phase 3.4 — `roomManager` was retired. Active games are now `Table` rows in Postgres (`previewState Json` + `seats Json`), so game state survives deploys by design. Socket reconnection after a brief drop re-joins the table room via the `TableDetailPage` flow. The scenario this item was guarding against no longer exists.

Residual concern worth tracking separately: when a socket drops mid-game, `useGameSDK` needs to rebind the move stream — today there's a short window where a move could be emitted to a disconnected socket. This is a socket-reconnect concern, not a state-persistence one, and is better filed as a gameplay-robustness task if it ever surfaces.

□ Configurable Guide — obsolete (premise gone)

Status: This item was written against the old iframe-based `public/getting-started.html` + 9-balloon SVG layout. That page no longer exists — it was retired during the Phase 2 nav restructure and the Phase 3.3 Guide panel rebuild. The Guide is now a React drawer (`landing/src/components/guide/GuidePanel.jsx`) with a journey card, slots grid, and notifications feed. Any future "configurable guide" work would be a fresh design against the new shell, not a continuation of this item. Leaving the original text below for archaeology:

What: Let users personalize the Getting Started guide through a "Configure Guide" panel in Settings. Three layers of configuration, in increasing complexity:

1. **Arrow toggle** — a switch to show or hide the dashed connector arrows between balloons. Some users find them helpful for understanding the progression; returning users who use the guide as a launcher find them visual noise.
2. **Balloon count** — a slider or stepper (1–9, the current maximum) controlling how many balloon positions are shown. Fewer balloons means a less cluttered guide focused on the actions the user actually uses. Hidden positions render empty

— the layout stays fixed so the guide doesn't reflow.

3. **Balloon assignment** — a drag-and-drop configurator where the user picks which function occupies each position. A palette lists all available destinations (Play, Gym, Puzzles, Rankings, Stats, Profile, About, FAQ, Settings, plus the Feedback and Have Fun easter eggs). The user drags a destination from the palette onto a slot in a miniature preview of the guide layout. The resulting assignment is saved and the guide renders accordingly.
4. **Presets** — 3–4 named configurations selectable with a single click, shown at the top of the Configure Guide panel before the manual controls. Selecting a preset populates the arrow toggle, balloon count, and slot assignments all at once; the user can then fine-tune from there. Candidate presets:
    - **Default** — the current fixed layout (all 9 balloons, arrows on, original assignments). Restores the out-of-the-box experience.
    - **Onboarding** — arrows on, all balloons visible, ordered as a learning path (FAQ → Play → Training Guide → Create Bot → Train → Compete).
    - **Launcher** — arrows off, 5–6 balloons showing only the most-used destinations (Play, Gym, Leaderboard, Profile, Puzzles). Optimized for returning users who treat the guide as a quick-action menu.
    - **Minimal** — arrows off, 3 balloons (user-chosen or defaulting to Play, Gym, Profile). Maximum signal, minimum clutter.

Balloon actions beyond simple navigation:

Each balloon in the palette would be associated with an *action*, not just a URL. An action is a small descriptor like `{ to: '/profile', open: 'bots' }` or `{ to: '/gym', focus: 'model-name' }`. When the user clicks the balloon, the guide posts the action to the parent via `postMessage`; the parent closes the modal and calls React Router's `navigate(to, { state: action })`. The destination page reads `location.state` on mount and performs the side effect — opening an accordion, scrolling to a section, setting focus on an input, pre-selecting a tab, etc.

This means the palette of available destinations is really a palette of *actions*, each with a label, an emoji, a destination route, and an optional UI side effect. Examples:

- **Play** → `/play` (no side effect)
- **Train a bot** → `/gym` + open the training panel
- **My Bots** → `/profile` + open the My Bots accordion
- **Leaderboard** → `/leaderboard` (no side effect)
- **Create a bot** → `/profile` + open the My Bots accordion + focus the Create New Bot input
- **Puzzles** → `/puzzles` (no side effect)
- **Settings** → `/settings` (no side effect)
- **FAQ** → `/faq` (no side effect)

This approach requires that the destination pages handle incoming `location.state` gracefully — if no state is present, they render normally; if state carries an `open` or `focus` key, they apply it on mount. It also means the current `<a target="_top">` implementation in the guide HTML must be replaced with `onclick` handlers that `postMessage` the action instead, since `<a>` tags can only carry a URL.

Current guide architecture and the key constraint:

The guide is a self-contained static HTML file (`/public/getting-started.html`) rendered in an iframe inside `GettingStartedModal`. The parent React app communicates with it via URL params (`?hint=faq`) and `postMessage`. The guide currently has 9 balloon positions at fixed SVG coordinates and 6 dashed arrow paths.

Making the guide configurable means the iframe must receive a config object and render dynamically rather than statically. Two approaches:

- **Pass config via postMessage (lower effort, preserves current architecture):** The parent serializes the user's guide config and sends it to the iframe after load (the guide already fires `getting-started-ready` to signal it's listening). The guide JS reads the config and shows/hides arrows, shows/hides balloon slots, and swaps each slot's emoji, label, and `href`. The drag-and-drop configurator lives entirely in the React Settings page — it never needs to be inside the iframe.
- **Convert guide to a React component (higher effort, cleaner long-term):** Remove the iframe and rewrite the SVG as a React component that reads guide config directly from the prefs store. No postMessage coordination needed. Loses the ability to link to the guide standalone, but makes all three config layers straightforward React state.

The postMessage approach is the right starting point — it extends the existing communication channel without a rewrite.

Persistence: Guide config is a small JSON blob (arrow visibility, balloon count, slot assignments) stored as a new field in user preferences — same pattern as `showGuideButton`, persisted via `api.users.updatePreferences` and loaded at sign-in via `api.users.getHints`.

What it would take: - **Schema:** add a `guideConfig` JSON column to the user preferences table. Default: arrows on, all 9 balloons, current fixed assignments. - **Settings UI:** a "Configure Guide" section below the existing Guide button toggle — arrow switch, balloon count stepper, and a drag-and-drop canvas showing the 9 slot positions with a destination palette beside it. - **Guide HTML:** replace hardcoded balloon content with a JS renderer that reads config from the `postMessage` payload and builds SVG elements dynamically. Arrow `<path>` elements toggled by CSS class; balloon `<a>` elements generated from the slot assignment array. - **`GettingStartedModal`:** after the iframe fires `getting-started-ready`, post the saved guide config to it.

Complexity: Medium-to-large (~3–4 days total). Arrow toggle alone is small (~2 hours). Balloon count adds half a day. The drag-and-drop configurator UI, the dynamic SVG renderer in the guide HTML, and schema/persistence together account for most of the estimate.